Luke Matsunaga

Outline of this report:

- Cover page: Page 1
- Report: Pages 2-5
- Sample problem of depth 4: Page 6-7
- Sample problem of depth 20: Page 8-9
- Source code : https://github.com/ActualCookie88/8_Puzzle_Solver

The 8-Puzzle

Luke Matsunaga

Introduction

The 8-puzzle is a 3x3 sliding puzzle that consists of 8 smaller tiles and an empty space. The empty space allows for the movement of a nearby tile, which creates a new empty space when a tile is moved; the number of tiles remains the same, but the arrangement of the tiles can be changed. Each tile has a distinct number, from 1 to 8, and the goal of the puzzle is to move each tile such that they are numbered in order from left to right, and up to down, with the empty space in the bottom right corner, as shown in *Figure 1*.

Figure 1: Example of an 8-puzzle from the internet

This puzzle seems simple to solve to most humans, but for computational purposes we must consider the cost and solutions of brute-force checking every reachable state. Since the search space of this puzzle can grow exponentially with depth, we need search strategies, heuristic in nature, to accomplish more informed search decisions.

In this project I use three search strategies: Uniform Cost Search, A* with the Misplaced Tile Heuristic, and A* with the Manhattan Distance Heuristic. I chose to use C++, and the full code can be found via the GitHub link provided above.

Uniform Cost Search

According to the instructions in the prompt given by Dr. Keogh, Uniform Cost Search is just A* with h(n) hardcoded to equal zero.[1] This algorithm works by expanding upon, or enqueuing, the cheapest node which is determined by the cost of the path g(n), or the number of moves taken since the initial position. In the special case of the 8-puzzle, Uniform Cost Search is equivalent to Breadth-First Search, since every possible move has the same cost. This algorithm is complete, though not necessarily optimal for increased depths due to the exponential nature of the 8-puzzle.

Misplaced Tile Heuristic (A*)

This algorithm is A* with the Misplaced Tile Heuristic, whose heuristic h(n) is determined by counting the number of tiles that are misplaced, or not in their goal position. For example, if two tiles are not in their goal position, regardless of their

[1] *Project_1_The_Eight_Puzzle_CS_170_2026 (by Dr. Eamonn Keogh, 2026)*

distance to their goal positions, then h(n) = 2. The total cost f(n) is calculated by adding g(n) and h(n), where g(n) equals the number of moves taken since the initial position. This heuristic never overestimates the cost to reach the goal state because it considers each misplaced tile, making it admissible.

Manhattan Distance Heuristic (A*)

Finally, this algorithm, A* with the Manhattan Distance Heuristic, determines the heuristic h(n) by summing the distances of each tile from their respective goal position, taking into account both column and row distance.

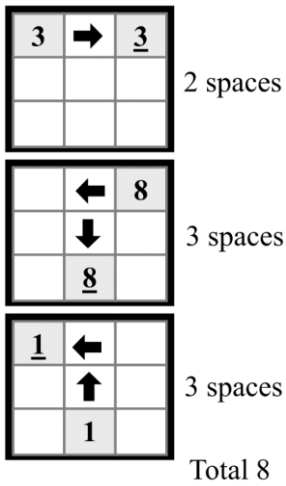

Figure 2: Illustration of Manhattan distances for tiles 1, 3, and 8. Adapted from "3__Heuristic Search" by E. Keogh, 2026, Course slides, UCR

In *Figure 2*, the 3 tile is two spaces away from its goal position, the 8 tile is 3 spaces away from its goal position, and the 1 tile is 3 spaces away from its goal position, so then h(n) = 8. This heuristic is more strongly informed, and thus more efficient, than the Misplaced Tile Heuristic, and allows for less node expansions.

Performance of Algorithms on Different Depths

Figure 3, shown below, shows various start state arrangements of the 8-puzzle sorted by solution depth, as provided by Dr. Keogh. The solution depth is the minimum number of moves it takes to reach the goal state. The leftmost arrangement of depth 0 shows the goal

state or solved puzzle, as each number is in numerical order and the empty space is in the bottom-right tile.

Figure 3: Samples of possible arrangements of the 8-puzzle. Adapted from "Project_1_The_Eight_Puzzle_CS_170_2026" by E. Keogh, 2026, Course slides, UCR

Figure 4 illustrates the performance of the three heuristics for each of the above solution depths. The Manhattan Distance Heuristic is the most efficient among the three heuristics, as it consistently expands fewer nodes as the depth increases drastically. The Misplaced Tile Heuristic performs well when compared to the Uniform Cost Search algorithm, which expands exponentially as depth increases.

As solution depth increases, the difference of performance between the three algorithms changes significantly. Uniform Cost Search explores every node until reaching the furthest depth, which explains an exponential behavior over large depths.

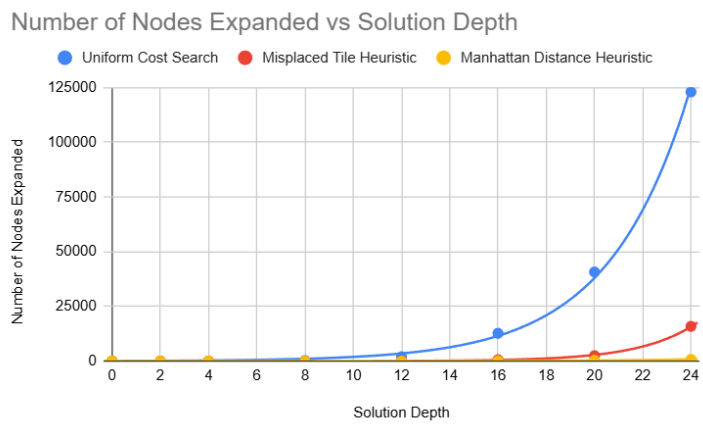

Figure 4: Comparison of the 3 heuristics on the 8-Puzzle Problem

The Misplaced Tile Heuristic expands states that seem closer to the goal state, through computation of g(n) and h(n). However, this heuristic does not compute the positions of tiles relative to their goal distance, so some arrangements may possess the same heuristics.

The Manhattan Distance Heuristic performs the best among the three and can be considered an improvement from the Misplaced Tile Heuristic since the heuristics of the various arrangements are more precise.

Conclusion

In this project, three search strategies were applied to the 8-puzzle problem, including Uniform Cost Search, A* with Misplaced Tile Heuristic, and A* with Manhattan Distance Heuristic. While each algorithm completed their respective solutions, their optimality differed greatly over longer depths.

Uniform Cost Search expanded the highest number of nodes for any number of depths greater than zero, as its heuristic was nonexistent. The Misplaced Tile Heuristic was

much more of an improvement in efficiency compared to the previous algorithm, but is ultimately dominated by the Manhattan Distance heuristic, which performed best. This strategy utilized a more consistent approach which reduced the required number of nodes to expand in depth tested.

The results of this assessment demonstrate the importance of heuristics in reducing unnecessary time and resources in terms of search methods. Furthermore, as shown through both A* heuristics, the existence as well as accuracy of the heuristic itself can improve search efficiency to a considerable extent.

Sample problem of depth 4:

////////////

[0 5 6]

[4 7 8]

Expanding best node: g(n)=2 h(n)=2 f(n)=4

[1 2 3]

[4 5 6]

[0 7 8]

Expanding best node: g(n)=3 h(n)=1 f(n)=4

[1 2 3]

[4 5 6]

[7 0 8]

Expanding best node: g(n)=4 h(n)=0 f(n)=4

[1 2 3]

[4 5 6]

[7 8 0]

PUZZLE SOLVED USING MISPLACED TILE HEURISTIC!

Solution Depth: 4

Nodes Expanded: 4

Max Queue size: 6

Sample problem of depth 20:

////////////

[1 2 3]

[4 5 6]

[7 8 0]

PUZZLE SOLVED USING MISPLACED TILE HEURISTIC!

Solution Depth: 20

Nodes Expanded: 2458

Max Queue size: 1556